

STEP-BY-STEP PROVISIONING

STEP 0 - Create Rekognition Collection

```
aws rekognition create-collection \  
--collection-id employees \  
--region eu-west-1
```

✓ (StatusCode 200 )

STEP 1 - Create S3 Buckets

1. Employee bucket

- Name: samuel-employee-images

2. Visitor bucket

- Name: samuel-visitor-images

✓ Block public access

✓ Enable versioning (recommended)

STEP 2 - Create DynamoDB Table

- Table name: employee
- Partition key: rekognitionid (String)

✓ On-demand capacity

✓ Default settings OK

STEP 3 - Create IAM Role for Lambda

Attach policies:

- AmazonS3FullAccess
- AmazonRekognitionFullAccess
- AmazonDynamoDBFullAccess

- CloudWatchLogsFullAccess

STEP 4 - Create Registration Lambda

Name:

employee-registration

Runtime:

Python 3.14

Logic:

```
rekognition.index_faces()  
dynamodb.put_item()
```

STEP 5 - Add S3 Trigger

- Bucket: employee-images
- Event: PUT

STEP 6 - Create Authentication Lambda

Name:

employee-authentication

Logic:

```
rekognition.search_faces_by_image()  
dynamodb.get_item()
```

STEP 7 - Create API Gateway

Steps:

1. Create REST API
2. Create resource: /authenticate
3. Create POST method
4. Integrate with Lambda

5. Enable CORS
6. Deploy (stage: dev)

STEP 8 - Connect API Gateway to Lambda

Add permission:

```
aws lambda add-permission \  
--function-name employee-authentication \  
--statement-id apigateway-test \  
--action lambda:InvokeFunction \  
--principal apigateway.amazonaws.com
```

STEP 9 - Build React Frontend

- ✓ Install NodeJS (done)
- ✓ Create app (Vite)
- ✓ Build upload form

Our frontend:

- Upload image
- Send to API Gateway
- Display result

- ✓ Confirmed working

STEP 10 - End-to-End Flow

Registration

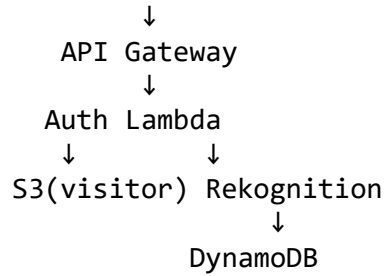
Upload → S3 → Lambda → Rekognition → DynamoDB

Authentication

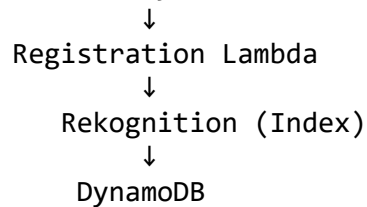
Upload → API Gateway → Lambda → Rekognition → DynamoDB → Response

FINAL ARCHITECTURE

[React Frontend]



[S3 employee bucket]



5. WHAT WE DID

- ✓ Correct service selection
- ✓ Proper event-driven architecture
- ✓ Working S3 trigger
- ✓ Clean separation of flows
- ✓ UI integrated with backend
- ✓ Logs verified (CloudWatch)

SUMMARY

We have essentially built a:

Serverless AI-powered Identity Verification System

This is Production-relevant

ARCHITECTURE EXPLANATION

High-Level Flow (from our diagram)

We built a **2-flow system**:

Flow 1: Employee Registration

1. Employee image uploaded → **S3 (employee bucket)**
2. S3 triggers → **Registration Lambda**
3. Lambda:
 - Calls **Rekognition (IndexFaces)**
 - Stores face metadata in **DynamoDB**

Flow 2: Authentication (Visitor / Employee)

1. User uploads image → **React frontend**
2. Frontend → **API Gateway**
3. API Gateway → **Auth Lambda**
4. Lambda:
 - Stores image in **Visitor S3 bucket**
 - Calls **Rekognition (CompareFaces / SearchFacesByImage)**
 - If match → fetch metadata from **DynamoDB**
 - Return result to frontend

Visual Architecture Breakdown

- **S3 (2 buckets)** → storage layer
- **Lambda (2 functions)** → compute logic
- **Rekognition** → AI engine
- **DynamoDB** → metadata store
- **API Gateway** → entry point
- **React App** → UI

Evidence from our files

✓ S3 Buckets Created

- samuel-employee-images
 - samuel-visitor-images
- 👉 Confirmed in S3 screenshots

✓ Lambda Functions

- employee-registration
- employee-authentication
- 👉 Confirmed in Lambda config pages

✓ S3 Trigger Working

- S3 → Lambda trigger added
- 👉 Seen in Lambda trigger config page

✓ DynamoDB Table

- Table: employee
- PK: rekognitionid
- 👉 Confirmed in DynamoDB UI

✓ CloudWatch Logs

- Lambda executions logged successfully
- 👉 Verified in logs

✓ API Gateway

- API created + deployed stages (dev, v1)
- 👉 Confirmed in API Gateway screenshots

✓ IAM Roles

- Lambda role with:
 - S3 access
 - Rekognition
 - DynamoDB 👉 Confirmed in IAM role setup

✓ React App

- Upload form working
- API interaction working
- Success/failure responses shown
- 👉 Confirmed in UI screenshots

